Certified Threshold Alerts over Incomplete Multi-Source Streams: Sound Abstention and Shared-Evidence Acquisition

Adishree Gupta
Dept. of CSE, PES University
Bengaluru, India
ORCID: 0009-0002-5909-135X

Akshat
Dept. of CSE, PES University
Bengaluru, India
ORCID: 0009-0008-2977-6298

Shylaja S S
Dept. of CSE, PES University
Bengaluru, India
ORCID: 0000-0003-2628-8973

***Abstract*—Monitoring systems evaluate threshold predicates over windowed aggregates ("alert if the 5-day traded volume of a stock exceeds 1.5× its norm"). Their inputs come from several sources and are routinely late, missing, or reported only approximately. Conventional engines fill the gaps with point estimates and emit alerts that may be wrong and are later silently reversed. We study the threshold-decision workload under interval-annotated evidence and make three contributions. (i) *Semantics and operator:* a three-valued certified alert (true / false / insufficient). Its certified outcomes hold in every completion consistent with the evidence, *provided the declared bounds contain the true values*. It is maintained incrementally over sliding windows and roll-ups in $O(\deg(i))$ per evidence change, with explicit detection of contradictory evidence. (ii) *Theory of acquisition:* under adversarial evidence, no proper subset of a decision's uncertain inputs can guarantee its resolution, so worst-case budgeted acquisition over shared evidence contains Densest-k-Subgraph and is NP-hard. For window-structured workloads, however, we give an exact strongly polynomial planner that maximises the number of alerts guaranteed to be settled. With industry roll-ups, an exact ILP shows that the planner's plan stays within 1% of the optimum. On the real panel it settles up to $1.9\times$ more than the best heuristic plan at low budgets, in about one second. (iii) *SEAQ*, an adaptive acquisition policy that scores each uncertain input by the share of uncertainty it removes, summed over all undetermined decisions it feeds. A controlled fan-out benchmark shows that its gains come from width normalisation rather than from sharing, and that they vanish at high fan-out. On a real panel of 398 Indian equities reported independently by two exchanges (NSE and BSE), and on a synthetic workload, SEAQ gives the largest area under the resolution-vs-budget curve in all nine settings (real, synthetic and heterogeneous-cost workloads, each under three missingness models). Its paired gain over the best baseline (myopic value of information) is 0.042–0.076 AUC on real data, and over a TRAPP-style widest-first policy it is 0.077–0.211. In a 460-day stream replay, certified alerts were wrong at emission 0.05% of the time versus 2.40% for imputation. We show that every such error requires a restatement or a violated declared bound. Certified alerts are emitted 0.84 days after window close on average, against 3.50 days for waiting for complete data. We also report where the approach does not help.**

I. Introduction

Motivating example. Market-surveillance pipelines, such as the capstone system that motivated this work (an NSE equity-surveillance pipeline built by one of the authors), raise volume alerts of the form

$$Q_k^{W,c}: \quad \sum_{t'=t-W+1}^{t} \left(v_{k,t'}^{\text{NSE}} + v_{k,t'}^{\text{BSE}}\right) > c \cdot W \cdot \text{med}_k$$

for each stock k, plus the same predicate rolled up to industries. The inputs arrive from two exchanges, often late; some come from secondary channels (vendor feeds, OCR of broadcast tickers) only as approximate readings. At window close a conventional engine replaces a missing input by an estimate (a median or last value) and emits a Boolean. In our replay that alert was wrong 18,001 times out of 749,892 emitted decisions (Section V-D), and 19,851 emitted decisions were later reversed with no indication of why.

The underlying database problem is broader than finance. It arises when (a) queries are *decisions* (HAVING/threshold predicates, alerts, SLA checks) rather than numbers; (b) evidence for an input is partial but *bounded*; and (c) inputs are *shared* by many decisions through overlapping windows, several thresholds and roll-ups. Two questions follow: *which decisions are already settled by the evidence?* and, for the rest, *which missing inputs should be fetched first when fetching costs money or time?*

What is known. Bounding query answers over incomplete data is well studied: certain answers and three-valued logic [1], [2], UA-DBs and AU-DBs that propagate attribute-level bounds [3], [4], and probabilistic databases [5], [6]. Choosing which values to refresh so that an aggregate meets a precision target was solved for a *single* query by TRAPP [7]. Probing for imprecise data [8] and oracle-based query-oriented cleaning [9] also address single queries or SPJ answers. None of these treats the *multi-decision* acquisition problem in which one input feeds many threshold decisions with different slacks. In that setting the per-query optimum is not the global optimum.

Core insight. The certified state of a SUM-threshold decision is determined only by two counters (L_d, U_d), so it can be maintained exactly and cheaply. Acquisition, however, has a sharp negative structure: under worst-case evidence a decision is guaranteed to be resolved only when *all* its uncertain inputs are known (Lemma 1). Myopic "resolve-in-one-probe" scores

are therefore mostly zero and mislead, which we also measure (Table II). A score that credits each input with the *fraction* of every undetermined decision's residual width it removes does better. It is simple and can be maintained over the sparse decision–input incidence.

Contributions (each checked in Section V):

1) A three-valued certified threshold operator over interval-annotated windowed aggregates with roll-ups. It has a soundness guarantee, narrowing-monotonicity, contradiction detection, and $O(\deg(i))$ incremental maintenance (Section II).
2) Lemma 1 (no partial worst-case certificate), Theorem 1 (worst-case budgeted acquisition with arbitrary sharing is NP-hard, also adaptively), and Theorem 2: a strongly polynomial exact optimiser for guaranteed-resolution planning on window workloads, and an exact ILP reference for roll-ups. On the real panel it settles up to $1.9\times$ more alerts than the best heuristic plan at low budgets (Sections III, V-G).
3) SEAQ, a shared-evidence acquisition policy, with ablations and a controlled fan-out benchmark that isolate where its gains come from, and where they disappear (Sections IV, V-F).
4) CQP, a DuckDB-backed prototype that keeps all state in SQL and implements the operator, retractions, SEAQ and the exact planner. It passes 16 correctness, equivalence and numerical-soundness tests, runs a deterministic end-to-end demo, and is benchmarked against full SQL recomputation on the real panel (Section V-H).
5) An evaluation on a real two-exchange panel and a synthetic workload under MCAR, late-arrival and adversarial (MNAR) missingness. It covers an end-to-end stream replay with restatements, and throughput against conventional and recompute engines (Section V). Code and the frozen development log are released with the submission (Section VIII).

II. Model and Certified Semantics

Definition 1 (Evidence)**.** *An* item i *has an unknown true value* $v_i \in \mathbb{R}_{\geq 0}$ *and evidence* $[\ell_i, h_i]$*. An item is* exact *if* $\ell_i = h_i$*,* approximate *if it carries a reading with a declared relative error* ϵ*, and* missing *if it carries only a declared domain bound* $[0, B_i]$*. The* soundness assumption *is* $v_i \in [\ell_i, h_i]$*. The width is* $w_i = h_i - \ell_i$*.*

Any modality whose extractor reports a bounded error (OCR digits with a checksum, a derived value with a tolerance, a vendor feed with a known deviation bound) reduces to this form. Unbounded corruption does not, and it is handled by contradiction detection below.

Definition 2 (Threshold decision)**.** *A decision* d *is a pair* (S_d, θ_d) *asking whether* $\sum_{i \in S_d} v_i > \theta_d$*. Let* $L_d = \sum_{S_d} \ell_i$*,* $U_d = \sum_{S_d} h_i$*,* $W_d = U_d - L_d$*. Its certified state is* T *if* $L_d > \theta_d$*,* F *if* $U_d \leq \theta_d$*, and* U *(insufficient) otherwise.*

A workload is a sparse incidence matrix $M \in \{0,1\}^{|D| \times |I|}$. Sliding windows of length W place an item in W consecutive decisions per threshold level, and roll-ups (industry sums) add further rows. COUNT reduces to SUM over indicator items. MIN/MAX use the analogous bounds $\max_i \ell_i$ and $\max_i h_i$, maintained with monotone deques; we evaluate SUM only.

Proposition 1 (Soundness and monotonicity)**.** *(a) If every item satisfies the soundness assumption, a decision in state* T *(*F*) is true (false) for the true values, and indeed for every completion consistent with the evidence. (b) Replacing* $[\ell_i, h_i]$ *by a sub-interval never changes a certified state. (c) If new evidence* $[\ell_i', h_i']$ *is disjoint from* $[\ell_i, h_i]$*, the old or the new evidence violates the soundness assumption.*

Proof. (a) $L_d \leq \sum v_i \leq U_d$. (b) Narrowing increases L_d and decreases U_d, so $L_d > \theta$ and $U_d \leq \theta$ are preserved. (c) A single v_i cannot lie in two disjoint intervals. □

Part (a) says certification is the *certain-answer* semantics [1], [2] specialised to aggregate thresholds; it is also what an AU-DB-style bound propagation would give [4]. We claim no novelty for (a)–(c). They fix the contract on which the operator and the acquisition problem rest. The *operator* reacts to a contradiction (case (c)) by flagging the item and *retracting* every emitted decision whose certification no longer holds, instead of silently flipping it.

Incremental maintenance. When item i changes from $[\ell, h]$ to $[\ell', h']$, the operator adds $\ell' - \ell$ to L_d and $h' - h$ to U_d for every d with $M_{di} = 1$. With a column-indexed incidence (or, for sliding windows, an implicit index range) this costs $O(\deg(i))$ time. The state test is $O(1)$ per touched decision. Storage is two counters per decision and two bounds per item, i.e. about twice a point engine. A recompute engine that re-sums each touched decision costs $O(\sum_{d \ni i} |S_d|)$ instead.

III. Theory of Acquisition

A *probe* of item i costs $c_i > 0$ and reveals v_i. Given budget B, which items should be probed?

Lemma 1 (No partial worst-case certificate)**.** *Let* d *satisfy* $L_d \leq \theta_d < U_d$ *(i.e. state* U*) and let* P *be a proper subset of its non-exact items. Then there are values* $v_i \in [\ell_i, h_i]$ *for* $i \in P$ *after which* d *is still in state* U*.*

Proof. Let $A = \sum_{i \in P} w_i < W_d$. Revealing values for P shifts L_d up by some $s \in [0, A]$ and U_d down by $A - s$, and every $s \in [0, A]$ is attainable by choosing the v_i. The new state is U iff $L_d + s \leq \theta_d < U_d - A + s$, i.e. $s \in (\theta_d - U_d + A,\ \theta_d - L_d]$. This interval has length $W_d - A > 0$. Its upper end $\theta_d - L_d \geq 0$, and its lower end $\theta_d - U_d + A < A$. Hence it meets $[0, A]$; $s = 0$ works when $\theta_d = L_d$ because then $\theta_d - U_d + A < 0$. □

So no non-adaptive plan short of probing *every* uncertain input of d can guarantee resolving d, however wide the probed items are. Guarantees are thus purely combinatorial. Exact items ($w_i = 0$) are irrelevant to the statement. A randomized search over 200,000 small instances (including zero-width items and ties $\theta_d = L_d$) found no counterexample to the constructive witness (`verify_theory.py`).

Theorem 1 (Hardness)**.** *Given decisions over shared items, unit costs and an integer k, choosing k items to maximise the number of decisions guaranteed (for all consistent values) to be resolved is NP-hard.*

Proof. Reduce from Densest-k-Subgraph [10]. Given $G = (V, E)$, create one item per vertex with evidence $[0, 1]$ and one decision per edge $\{u, v\}$ with $S_d = \{u, v\}$ and $\theta_d = 1$, so $L_d = 0 < 1 < 2 = U_d$. By Lemma 1, d is guaranteed resolved iff both endpoints are probed, and probing both resolves it. Hence the number of guaranteed-resolved decisions equals the number of edges induced by the chosen k vertices. □

Adaptivity does not help in the reduction: an adversary that answers 0.5 to every probe leaves each edge with an unprobed endpoint in state U ($L = 0.5 \leq 1 < 1.5 = U$). So the adaptive worst-case version is NP-hard on the same family; we verified the correspondence exhaustively on random 6-vertex graphs. The reduction uses arbitrary pairwise sharing. Pure sliding windows over one series have interval structure and are not covered by it. The next result shows that this structure makes the problem tractable.

Guaranteed-resolution planning (GR). Given budget B, choose probes so as to maximise the number of undetermined decisions that are resolved *for every consistent outcome*. The result is a plan whose payoff can be committed to before any data arrive: "with budget B we will settle at least $N^*(B)$ alerts". By Lemma 1, a decision is guaranteed iff all its uncertain items are probed. Partially probing a time slot of a series (one key-day with several sources) never helps, so slots are atomic with cost $c_t \geq 0$.

Theorem 2 (Exact GR for window workloads)**.** *Suppose every decision on key k covers a contiguous range of time slots of k (sliding windows of any lengths and any number of thresholds), and keys share no items. Then GR is solvable exactly in $O\big(\sum_k m_k^2\,|D_k| + |D|^2\big)$ arithmetic operations (additions and comparisons on costs), where m_k is the number of uncertain slots of key k and D_k its undetermined decisions. The bound holds for arbitrary nonnegative real (or binary-encoded rational) costs and budgets, so the algorithm is strongly polynomial.*

Proof. Fix key k with uncertain slots $u_1 < \cdots < u_m$ and sentinels u_0, u_{m+1}. Each undetermined decision maps to an index range $[p_d, q_d]$ of uncertain slots. It is guaranteed iff no *unprobed* uncertain slot lies in $[p_d, q_d]$, i.e. iff the range lies strictly between two consecutive unprobed indices. Any plan is therefore described by its increasing sequence of unprobed indices $0 = i_0 < i_1 < \cdots < i_r = m + 1$. Its cost is $\sum_\ell \mathrm{cost}(i_{\ell-1}, i_\ell)$ with $\mathrm{cost}(i, j) = \sum_{i<l<j} c_l$, and its value is $\sum_\ell G(i_{\ell-1}, i_\ell)$ with $G(i, j) = |\{d : i < p_d,\ q_d < j\}|$. Each guaranteed decision is counted exactly once, in the unique gap that contains its range. Both cost and value decompose over consecutive pairs. We index the table by *value*, which is an integer bounded by $|D_k|$, rather than by budget: $\mathrm{mc}[j][v] = \min_{i<j} \mathrm{mc}[i][v - G(i, j)] + \mathrm{cost}(i, j)$ is the minimum cost of a plan whose last unprobed index is j and that guarantees v decisions. This takes $O(m^2|D_k|)$ operations, with G tabulated in $O(m^2)$ from 2-D prefix counts. Keys are independent, so the global minimum cost $g(v)$ of guaranteeing v decisions is the min-plus convolution of the per-key tables. This costs $O(|D|^2)$ overall, and the answer is $\max\{v : g(v) \leq B\}$. No step depends on the magnitude of costs or of B. *Slot atomicity is without loss of optimality.* An uncertain item of slot t matters only to decisions containing t, and each such decision needs *all* uncertain items of t. Dropping the probed items of an incompletely probed slot therefore keeps the value and does not increase the cost. Item-level costs thus aggregate into $c_t = \sum$ of the slot's uncertain item costs, and zero-cost items are harmless. *Bit complexity:* every number the algorithm stores is a sum of at most m_k (or $\sum_k m_k$) input costs, so its encoding length exceeds that of the inputs by $O(\log \sum_k m_k)$ bits. Space is $O(m_k|D_k|)$ per key plus $O(|D|)$ for the convolution. □

Complexity by cost model. The budget-indexed variant of the same recurrence (our first implementation) is pseudo-polynomial, $O(\sum_k m_k^2 B)$. It is polynomial for unit or polynomially bounded integer costs (then $B \leq \sum c$), but not in the binary encoding of B. The value-indexed form removes this dependence. It also returns the whole cost-versus-guarantee frontier $g(\cdot)$ in one pass, which is what a budget planner needs. We distinguish three workload classes. In *Class A* (independent keys, any overlapping windows and thresholds), GR is strongly polynomial (Thm. 2). *Class B* (industry or laminar roll-ups over Class A) has open complexity; we solve it exactly by ILP and observe near-optimality of the Class-A plan only on the tested instances. In *Class C* (arbitrary sharing), GR is NP-hard (Thm. 1). Theorems 1 and 2 together draw the tractability line *inside* the monitoring workload. Overlapping windows of mixed lengths, repeated thresholds, and several sources per slot are strongly polynomial. Arbitrary sharing is NP-hard. Industry roll-ups lie in between: their decisions span the slots of many keys over a time interval (a laminar key family times intervals), which breaks the per-key decomposition. We do *not* settle their complexity. Instead we solve them exactly with a 0-1 ILP whose group-day variables $z_{g,t} \leq x_{k,t}$ keep the model linear in the size of the incidence (Section V-G). We checked both implementations against exhaustive search. The budget-indexed one (`wcdp.py`) matched on 3,000 random integer-cost instances. The value-indexed one (`wcdp_sp.py`) matched on 2,000 real-cost instances, on the integer cases, and on 300 two-key combinations, and the ILP matched on 200 arbitrary-sharing instances. A further audit used *exact rational arithmetic* and brute force over *individual source items*, which also tests slot atomicity. It covered 1,500 instances, including zero-cost items, zero budgets, empty decision sets, and identical and nested ranges (`verify_exact.py`). There were no mismatches. Random testing is evidence, not proof; the proof above is the argument. The dynamic program itself uses a standard gap-decomposition technique. What we contribute is the GR formulation for certified decisions and the tractability

boundary it reveals. Densest-k-Subgraph is also hard to approximate under standard complexity assumptions (see [10] and follow-up work). We therefore do not pursue worst-case guarantees. Instead we optimise the *empirical* resolution curve with an adaptive policy and measure it. Adaptive acquisition for a single linear threshold function is a stochastic Boolean function evaluation problem [11]. The multi-decision, shared-input version studied here is what our policy targets.

IV. SEAQ: Shared-Evidence Acquisition

For an undetermined decision d define the gaps $g_d^L = \theta_d - L_d$ and $g_d^U = U_d - \theta_d$, with $W_d = g_d^L + g_d^U$. Two natural per-input signals exist:

- *One-probe resolution probability.* If $v_i \sim \mathrm{Unif}[\ell_i, h_i]$, probing i resolves d with probability exactly $P_1(d,i) = \big(\max(0, w_i - g_d^L) + \max(0, w_i - g_d^U)\big)/w_i$, since d becomes T iff $v_i - \ell_i > g_d^L$ and F iff $h_i - v_i \geq g_d^U$. This is the myopic value-of-information (VOI) score.
- *Progress share.* w_i/W_d, the fraction of d's residual uncertainty that probing i removes. By Lemma 1, resolution needs the residual width to collapse below the gap, so progress counts even when $P_1 = 0$.

SEAQ probes, in batches of b, the items with the largest

$$\mathrm{score}(i) = \frac{1}{c_i} \sum_{d:\, M_{di}=1,\ \mathrm{state}(d)=\mathsf{U}} \frac{w_i}{W_d}. \tag{1}$$

After each batch the revealed values update (L_d, U_d) incrementally and resolved decisions drop out of the sum. The sum over decisions is what exploits *sharing*: an input feeding a stock window, its industry roll-up and two threshold levels is credited for all of them. The normalisation by W_d prevents one wide missing input from dominating, which is the failure mode of a widest-first rule.

Design history (disclosed). Our a-priori design was $\sum_d [P_1(d,i) + \lambda w_i/W_d]/c_i$ with $\lambda = 0.5$. On development data only (synthetic seed 1; DEV_LOG D1) the area under the curve rose monotonically with λ, and $\lambda \to \infty$ was best. We froze Eq. (1) before any real-data run and keep the original form as an ablation (*SEAQ+VOI*).

Cost. One scoring pass touches the non-zeros of M restricted to undetermined decisions, nnz_U. It costs $O(\mathrm{nnz}_\mathsf{U})$ with a sparse bin-count, plus $O(\deg(i))$ per revealed item. A full run to complete resolution costs $O(\mathrm{nnz}_\mathsf{U} \cdot n_{\text{probes}}/b)$. On our largest episode (1672 decisions, 15920 items) a complete SEAQ run took about one second in Python/NumPy.

V. Evaluation

A. Workloads and protocol

Real. Daily traded volumes of 398 NIFTY-500 equities that are reported independently by NSE and BSE (official bhavcopy files, 580 trading days from July 2024). Industries come from the NIFTY-500 constituents file. Each stock-day therefore gives two items, one per exchange. The files were downloaded for a separate study of feed reconciliation; we reuse only the exchange files, and none of that study's methods or claims. The first 120 days calibrate thresholds (med_k) and domain bounds ($B_i = 3\times$ the calibration maximum of that series). They are never evaluated. **Synthetic.** 400 keys, 20 groups, AR(1) log-volumes with 3% spikes, and the same schema.

Decisions per episode. For an evaluation day t: windows $W \in \{5, 20\}$, threshold multiples $c \in \{1.0, 1.5\}$, at stock and industry level. This gives 1672 decisions over 15920 items on the real panel.

Evidence scenarios. *MCAR*: each item is missing w.p. 0.3. *LATE*: missing w.p. $0.9 \cdot 0.7^{\text{lag}}$, so recent days are mostly missing. *MNAR (adversarial)*: missing w.p. 0.6 if the value is a spike ($> 1.5\times$ median), else 0.15. Non-missing items are approximate w.p. 0.3, with a reading $r = v(1+\eta)$, $\eta \sim U(-0.1, 0.1)$, and the sound interval $[r/1.1, r/0.9]$. Missing bounds can be *violated* when a true value exceeds B_i; we count these violations instead of hiding them.

Acquisition protocol. The budget is expressed as a fraction of WAIT, the cost of probing every uncertain input of every undetermined decision. We report $\mathrm{AUC} = \frac{1}{51}\sum_{j=0}^{50} R(0.01j)$, where $R(b)$ is the fraction of initially undetermined decisions resolved after spending b·WAIT. This is the *mean* resolved fraction over budgets $b \in [0, 0.5]$; it lies in $[0, 1]$ and is not an unnormalised integral. We also report the resolved fraction at 10% budget (R@10), the budget needed for 90% resolution (C90), and a paired bootstrap 95% CI (2000 resamples over episodes) for the AUC difference against SEAQ. There are 8 real episodes and 12 synthetic episodes per scenario, with fresh evidence draws. Batch size $b = 8$ and unit costs, unless stated otherwise.

Baselines. *Random*. *Widest* (largest w_i/c_i; the TRAPP-style per-item width reduction of [7] applied to the union of decisions). *Degree* (number of undetermined decisions fed). *Local*: per-query probing in the spirit of [7], [8], which repeatedly picks the undetermined decision with smallest normalised gap and probes its widest input. *Myopic VOI*: $\sum_d P_1(d,i)/c_i$. Ablations: *SEAQ+VOI* (the a-priori design) and *SEAQ-noshare* (max instead of sum over decisions).

B. Certification at zero budget vs. imputation

Table I shows the semantics at work. With 30% missing inputs the evidence alone settles only about a fifth of the decisions, and none of these was wrong in any episode, despite

TABLE I
Zero-budget certification vs. point imputation (real panel, mean over episodes). Imputation decides every decision; its error is reported overall, on decisions our operator leaves undetermined, and on those it certifies.

	MCAR	LATE	MNAR
Certified fraction (%)	23.8	22.1	20.3
Certified but wrong (total)	0	0	0
Bound violations (total items)	110	62	228
Imputation error, all (%)	7.2	8.2	13.7
on undetermined (%)	9.4	10.5	17.2
on certified (%)	0.0	0.0	0.0

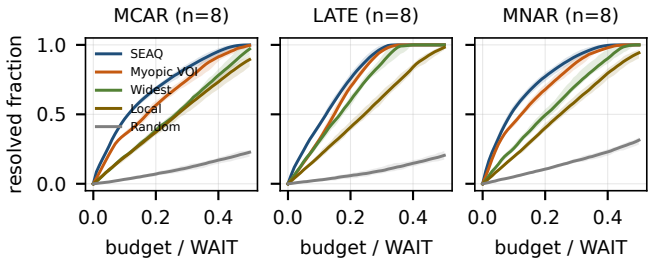


Fig. 1. Resolved fraction of initially undetermined decisions versus acquisition budget on the real panel (mean over n episodes, band = min–max over episodes), plotted from the stored per-episode curves.

a few violated domain bounds. On the rest, imputation errs 9.4% (MCAR) to 17.2% (MNAR) of the time and reports these errors as ordinary Booleans. On the certified subset imputation was also correct (0%), so certification adds no accuracy there. Its value is that it *identifies* which decisions need evidence. Missingness correlated with spikes (MNAR) nearly doubles imputation error, because the missing values are exactly the ones that decide the alert.

C. Acquisition

Statistical unit. The unit is an episode: one evaluation day with its own evidence draw. Decisions within an episode are not independent, so all CIs resample episodes, not decisions. The 8 (real) or 12 (synthetic) episode days are distinct, but they come from one market panel, so the CIs quantify variation over days of *this* panel. They are not evidence of generalisation to other markets. In every baseline comparison SEAQ wins on *every* episode (exact one-sided sign test $p \leq 0.0039$; `results/stats_signtest.json`). The 9 settings of Table II are confirmatory with respect to the frozen SEAQ (DEV_LOG D1); all later policy variants are labelled exploratory. Table II gives the main result, and Fig. 1 shows the underlying curves. SEAQ has the highest AUC in every cell, and every paired CI excludes zero. Three observations stand out. (1) *Myopic VOI is the strongest baseline but is beaten consistently.* Its gap to SEAQ is 0.042–0.076 AUC on real data, and every paired CI excludes zero. Adding the VOI term to SEAQ *hurts* (SEAQ+VOI). This is the empirical face of Lemma 1: one-probe resolution probabilities are zero for most inputs and push probes toward decisions that happen to be nearly resolved. (2) *Normalisation matters more than sharing.* Widest-first, the TRAPP-style rule, trails by 0.077–0.211, and Local per-query probing trails by more. In contrast, removing sharing (SEAQ-noshare) costs only 0.006–0.019 AUC. The gain from sharing is real (CIs exclude zero) but *small* on this workload, in which each input feeds at most 8 decisions. Section V-F tests whether it grows with fan-out; it does not. (3) *The ordering is the same on real and synthetic data*, under adversarial MNAR missingness, and with heterogeneous probe costs (Real-het columns), where SEAQ again has the highest AUC in all three scenarios. A handful of acquisition runs certified a decision wrongly (at most 3 per scenario across all policies). Each was traced to a violated domain bound, so the soundness assumption, not the policy, failed.

D. End-to-end stream replay

We replay the real panel as a stream of 476,479 evidence events: for each item, an on-time exact report (40%), a late exact report (30%, geometric delay), or an approximate report followed later by the exact value (30%). In addition, 721 items (0.2%) are later *restated* by $\times 0.6$ or $\times 1.5$ to model trade busts and corrections. Ground truth is the final restated value. Eight decision families (2 windows $\times$ 2 thresholds $\times$ stock/industry) are evaluated every day. That is 749,892 decisions. Table III shows the trade-off that motivates the operator. POINT is always on time but wrong 2.40% of the time, and it reverses 19,851 decisions without explanation. WAIT is almost never wrong but arrives 3.50 days late on average. CERT emits most decisions on time (62.9%), with a mean delay of 0.84 days. Its residual errors (397) are caused by restatements and violated bounds, i.e. by *unsound* evidence, which no evidence-based method can anticipate. Every CERT reversal is attributable to a detected contradiction. Incremental maintenance of two bounds runs at 71.3k events/s versus 135.5k for a point engine (about 1.9$\times$ overhead) and 2.9k for recompute-on-change. All numbers are single-threaded Python. Absolute throughput is not the claim; the ratios are.

E. Where do certified errors come from?

Proposition 1 says certified errors *require* unsound evidence. Table IV tests this by removing the two sources of unsoundness one at a time. Without restatements, the only errors left are those from items whose true value exceeds the declared $3\times$ bound. Raising the bound to $10\times$ removes most violations and the corresponding errors, but it costs on-time coverage, because wider bounds certify less. With restatements on, WAIT is wrong about as often as CERT: a restated input invalidates any decision taken before the restatement, however complete the data looked. Contradiction detection (Proposition 1(c)) turns each such case into a flagged retraction, but it cannot prevent the initial error. When both sources of unsoundness are removed ($1000\times$ bounds, no violating items, no restatements), CERT made *zero* wrong certifications, but on-time emission fell from 62.9% to 53.4%. Every observed error is therefore attributable to an explicit assumption violation. Retraction repairs an error *after* it happened and does not prevent it. The guarantee is therefore exactly as strong as the declared bounds, and choosing the bound trades coverage against safety.

F. Controlled fan-out: when does sharing matter?

The real workload gives each input a fan-out of at most 8. To test whether sharing matters more at higher fan-out, we built a controlled benchmark that varies only the sharing structure. It has 1024 decisions of 32 items each, and every item feeds exactly f decisions in a random regular design. Missingness is 50%, missing items carry the bound $[0, 4\mu]$, item values are Gamma-distributed with per-item mean μ, and thresholds are $\theta_d = |S_d|\,\mu{\cdot}U(0.9, 1.1)$, i.e. within $\pm 10\%$ of the

TABLE II

RESOLUTION VS. BUDGET (BUDGET = FRACTION OF WAIT). AUC = MEAN RESOLVED FRACTION OVER BUDGETS (51-POINT GRID ON) $[0, 0.5]$ (HIGHER IS BETTER), C90 = BUDGET FOR 90% RESOLUTION (LOWER IS BETTER), AND PAIRED BOOTSTRAP 95% CI OF AUC(SEAQ)−AUC(POLICY) OVER EPISODES. REAL: NSE+BSE PANEL, 8 EPISODES/SCENARIO; SYNTHETIC: 12 EPISODES/SCENARIO; REAL-HET: HETEROGENEOUS PROBE COSTS (PRIMARY-EXCHANGE ITEM 3, SECONDARY 1), 12 EPISODES.

		Real (unit cost)			Synthetic (unit cost)			Real-het (cost 3/1)		
Scen.	Policy	AUC	C90	ΔAUC [95% CI]	AUC	C90	ΔAUC [95% CI]	AUC	C90	ΔAUC [95% CI]
MCAR	Random	0.099	0.951	+0.596 [0.585,0.605]	0.109	0.947	+0.570 [0.565,0.575]	0.105	0.944	+0.570 [0.562,0.578]
MCAR	Widest (TRAPP-style)	0.484	0.462	+0.211 [0.204,0.218]	0.466	0.485	+0.213 [0.209,0.217]	0.409	0.511	+0.266 [0.263,0.269]
MCAR	Degree	0.485	0.627	+0.210 [0.204,0.216]	0.478	0.669	+0.201 [0.197,0.205]	0.255	0.755	+0.421 [0.414,0.425]
MCAR	Local (per-query)	0.462	0.504	+0.234 [0.224,0.242]	0.453	0.541	+0.226 [0.223,0.230]	0.412	0.552	+0.263 [0.255,0.271]
MCAR	Myopic VOI	0.620	0.386	+0.076 [0.071,0.080]	0.615	0.422	+0.064 [0.062,0.066]	0.582	0.450	+0.093 [0.089,0.098]
MCAR	**SEAQ**	0.695	0.352	–	0.679	0.379	–	0.675	0.376	–
MCAR	SEAQ+VOI (abl.)	0.643	0.372	+0.052 [0.048,0.056]	0.638	0.405	+0.041 [0.039,0.043]	0.611	0.432	+0.064 [0.061,0.068]
MCAR	SEAQ-noshare (abl.)	0.689	0.359	+0.006 [0.004,0.007]	0.669	0.389	+0.010 [0.009,0.011]	0.662	0.392	+0.013 [0.011,0.014]
LATE	Random	0.086	0.933	+0.651 [0.646,0.656]	0.091	0.922	+0.636 [0.630,0.642]	0.094	0.925	+0.619 [0.613,0.625]
LATE	Widest (TRAPP-style)	0.659	0.313	+0.077 [0.073,0.081]	0.651	0.317	+0.075 [0.073,0.077]	0.606	0.343	+0.108 [0.104,0.112]
LATE	Degree	0.553	0.439	+0.183 [0.175,0.191]	0.545	0.462	+0.182 [0.178,0.186]	0.296	0.567	+0.417 [0.400,0.433]
LATE	Local (per-query)	0.512	0.434	+0.224 [0.217,0.231]	0.522	0.437	+0.205 [0.201,0.209]	0.483	0.462	+0.231 [0.224,0.237]
LATE	Myopic VOI	0.694	0.273	+0.042 [0.039,0.045]	0.685	0.285	+0.042 [0.041,0.043]	0.648	0.308	+0.066 [0.062,0.070]
LATE	**SEAQ**	0.736	0.257	–	0.727	0.269	–	0.714	0.281	–
LATE	SEAQ+VOI (abl.)	0.706	0.266	+0.030 [0.028,0.033]	0.697	0.279	+0.030 [0.029,0.031]	0.668	0.298	+0.045 [0.043,0.048]
LATE	SEAQ-noshare (abl.)	0.717	0.274	+0.019 [0.018,0.020]	0.706	0.287	+0.021 [0.019,0.023]	0.691	0.300	+0.023 [0.021,0.025]
MNAR	Random	0.142	0.937	+0.612 [0.602,0.623]	0.143	0.938	+0.554 [0.548,0.561]	0.152	0.930	+0.567 [0.556,0.577]
MNAR	Widest (TRAPP-style)	0.578	0.398	+0.175 [0.167,0.183]	0.510	0.460	+0.188 [0.185,0.191]	0.482	0.462	+0.237 [0.227,0.248]
MNAR	Degree	0.499	0.597	+0.254 [0.245,0.264]	0.475	0.676	+0.222 [0.219,0.225]	0.257	0.745	+0.462 [0.452,0.470]
MNAR	Local (per-query)	0.493	0.464	+0.260 [0.251,0.269]	0.464	0.530	+0.233 [0.228,0.238]	0.440	0.518	+0.279 [0.269,0.288]
MNAR	Myopic VOI	0.702	0.314	+0.051 [0.049,0.054]	0.647	0.394	+0.051 [0.049,0.052]	0.648	0.389	+0.071 [0.069,0.073]
MNAR	**SEAQ**	0.753	0.283	–	0.698	0.356	–	0.719	0.323	–
MNAR	SEAQ+VOI (abl.)	0.721	0.308	+0.032 [0.029,0.035]	0.666	0.384	+0.031 [0.030,0.032]	0.673	0.374	+0.046 [0.044,0.048]
MNAR	SEAQ-noshare (abl.)	0.744	0.293	+0.009 [0.007,0.010]	0.686	0.369	+0.012 [0.011,0.013]	0.708	0.334	+0.011 [0.010,0.013]

TABLE III

STREAM REPLAY ON THE REAL PANEL (ONE REPLAY, SEED 7). WRONG = WRONG WHEN FIRST EMITTED. DELAY = DAYS AFTER WINDOW CLOSE. REVERSALS = LATER CHANGES OF AN EMITTED DECISION; FLAGGED = REVERSALS ATTRIBUTABLE TO A DETECTED CONTRADICTION.

	POINT (impute)	WAIT (complete)	CERT (ours)
Wrong at emission (%)	2.40	0.05	0.05
Wrong at emission (count)	18,001	344	397
Mean delay (days)	0.00	3.50	0.84
Emitted on time (%)	100.0	3.0	62.9
Reversals	19,851	344	415
of which flagged	0	344	415
Throughput (k ev/s)	135.5	–	71.3 (recomp. 2.9)

TABLE IV

WHERE CERTIFIED ERRORS COME FROM (STREAM REPLAY, SEED 7). BOUND = DECLARED DOMAIN BOUND FOR UNREPORTED ITEMS (MULTIPLE OF CALIBRATION MAX). VIOLATING ITEMS = ITEMS WHOSE TRUE VALUE EXCEEDS THE BOUND.

Bound, restat.	Viol.	CERT err	on-time %	delay (d)	WAIT err
3×, restat. (main)	1364	397	62.9	0.84	344
3×, no restat.	1363	36	62.8	0.84	0
10×, restat.	183	364	57.8	1.00	344
10×, no restat.	183	7	57.7	1.00	0
1000×, no restat.	0	0	53.4	1.17	0

decision's expected sum. This is the formula implemented in `run_fanout.py`, and all 1024 decisions start undetermined. Decision count, decision size, missingness and threshold rule are fixed. The item pool necessarily shrinks as $D \cdot s/f$. Table V *refutes* the hypothesis that the benefit of sharing grows with fan-out. The gain of SEAQ over SEAQ-noshare peaks at $f = 2$–4 (at most $+0.006$) and is indistinguishable from zero for $f \geq 16$. The gain over myopic VOI is largest at $f = 2$ ($+0.031$) and shrinks to zero at $f = 32$. At $f = 1$ (no sharing) SEAQ, VOI and noshare coincide. More importantly, every policy degrades as f grows. Each decision then depends on inputs that are also uncertain for many other decisions, and with thresholds near the mean, almost all of a decision's uncertain inputs must be fetched (C90 approaches 1, as Lemma 1 predicts for adversarial cases). In that regime, simply fetching everything (WAIT) is as good as any policy. SEAQ's measured advantage therefore comes mainly from *width normalisation* (versus Widest and Degree) and from avoiding myopic VOI's zero scores at low-to-moderate fan-out. It does not come from exploiting sharing.

G. Exact guaranteed-resolution planning

Fig. 2 plots the guaranteed counts. We build a 40-day backlog of stock-level windows on the real panel ($W \in \{5, 20\}$, two thresholds, windows ending on each of the last 40 days; about 7.5k (LATE) and 33k (MCAR) undetermined decisions). We compare the exact plan of Theorem 2 with non-adaptive plans of the same cost. Table VI gives the answer. The best heuristic, a classic cheapest-completion greedy, reaches only 52–54% of the optimum at a 2% budget and 83–90% at 40%. Plans ordered by SEAQ or width reach at most 22%, because those scores spread probes across decisions instead of completing them. The exact optimiser solves the whole panel in about 0.7–1.3 s in Python. This is the capability the paper adds beyond bound propagation: an acquisition plan with a *guaranteed* and *provably maximal* number of settled alerts.

TABLE V

CONTROLLED FAN-OUT BENCHMARK: 1024 DECISIONS OF 32 ITEMS EACH, EVERY ITEM FEEDS EXACTLY f DECISIONS (5 INSTANCES PER f). MEAN AUC; LAST COLUMNS: AUC GAIN OF SEAQ OVER SEAQ-NOSHARE AND OVER MYOPIC VOI WITH MIN–MAX OVER INSTANCES, SEAQ C90, AND SEAQ RUNTIME.

f	items	Rand	Widest	Deg	VOI	noshare	SEAQ	Δnoshare	ΔVOI	C90	s
1	32768	0.011	0.013	0.012	0.316	0.316	0.316	+0.000 [+0.000,+0.000]	+0.000 [-0.001,+0.001]	0.838	2.5
2	16384	0.011	0.015	0.013	0.172	0.198	0.202	+0.005 [+0.002,+0.007]	+0.031 [+0.029,+0.032]	0.911	1.5
4	8192	0.013	0.015	0.015	0.102	0.121	0.127	+0.006 [+0.003,+0.009]	+0.025 [+0.023,+0.026]	0.931	0.5
8	4096	0.010	0.011	0.010	0.060	0.072	0.075	+0.003 [-0.000,+0.006]	+0.015 [+0.013,+0.016]	0.956	0.3
16	2048	0.010	0.010	0.009	0.034	0.042	0.043	+0.001 [-0.002,+0.004]	+0.009 [+0.006,+0.012]	0.969	0.1
32	1024	0.010	0.009	0.008	0.028	0.026	0.026	-0.000 [-0.004,+0.001]	-0.002 [-0.011,+0.003]	0.978	0.1

TABLE VI

GUARANTEED-RESOLUTION PLANNING ON THE REAL PANEL (STOCK-LEVEL WINDOWS, 40-DAY BACKLOG, 5 DAYS PER SCENARIO). ROW 1: OPTIMUM NUMBER OF DECISIONS GUARANTEED RESOLVED (EXACT DP, THM. 2), MEAN OVER DAYS. OTHER ROWS: FRACTION OF THE OPTIMUM ACHIEVED BY EACH NON-ADAPTIVE PLAN AT THE SAME BUDGET (BUDGET = FRACTION OF TOTAL UNCERTAIN-ITEM COST).

Scen.	Plan	2%	5%	10%	20%	40%
LATE	**Exact DP (count)**	527	1153	2040	3527	5755
	Greedy cheapest-completion	0.52	0.67	0.66	0.74	0.83
	Completion score	0.41	0.44	0.51	0.61	0.75
	SEAQ order	0.11	0.08	0.08	0.11	0.22
	Widest order	0.06	0.06	0.08	0.13	0.20
	Random	0.00	0.01	0.01	0.02	0.05
	decisions: 7475; exact DP time: 0.66 s					
MCAR	**Exact DP (count)**	1088	2584	4946	9369	17464
	Greedy cheapest-completion	0.54	0.58	0.71	0.86	0.90
	Completion score	0.27	0.23	0.22	0.23	0.44
	SEAQ order	0.04	0.03	0.03	0.04	0.08
	Widest order	0.06	0.06	0.06	0.07	0.11
	Random	0.00	0.00	0.01	0.01	0.05
	decisions: 32837; exact DP time: 1.31 s					

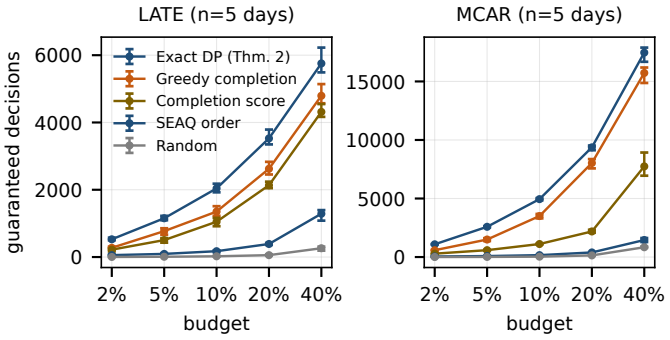


Fig. 2. Decisions guaranteed resolved versus budget: exact optimum (Thm. 2) and non-adaptive heuristic plans (mean over 5 days, bars = min–max). The exact curve is an optimum, not an estimate; the others are measured plans.

The two objectives call for different tools. SEAQ is the right policy for *expected* resolution (Table II); the exact plan is the right one for *commitments*.

With roll-ups. Table VII adds the industry roll-up decisions and solves the full problem exactly with the ILP. HiGHS proved optimality in 15–30 s on every instance. Two findings stand out. First, roll-up decisions are rarely worth guaranteeing: each needs every member's uncertain slots, so the optimum settles only a few dozen of the 380–610 roll-ups. Second, the plan of the window DP, which ignores roll-ups,

TABLE VII

GUARANTEED-RESOLUTION PLANNING WITH INDUSTRY ROLL-UPS (REAL PANEL, LATE, 40-DAY BACKLOG, 3 DAYS). ENTRIES: STOCK + INDUSTRY DECISIONS GUARANTEED (MEAN OVER DAYS), RATIO TO THE EXACT OPTIMUM IN PARENTHESES. THE ILP PROVED OPTIMALITY ON ALL 6 SOLVES IN 15–30 S; THE WINDOW DP (THM. 2) TAKES 1.1 S, THE VALUE-INDEXED DP WITH REAL-VALUED COSTS 0.75 S. MEAN DECISIONS: 7294 STOCK + 503 INDUSTRY.

Plan	5% budget	20% budget
Exact ILP (HiGHS)	1133+13 (1.000)	3438+41 (1.000)
Window DP, stock only	1141+3 (0.998)	3454+3 (0.994)
Greedy completion	759+3 (0.664)	2556+6 (0.737)
SEAQ order	110+3 (0.098)	411+4 (0.120)
Widest order	78+1 (0.070)	439+5 (0.128)

reaches at least 99% of the full optimum at about 1 s. The greedy completion plan reaches 66–76%, and SEAQ/width orders reach at most 14%. On this workload the tractable class therefore captures almost all of the achievable guarantee, and an exact ILP certifies the remaining gap. We expect the gap to grow when roll-ups dominate (small groups, few uncertain members), which we did not test.

Negative result: completion scores do not help expected resolution. Motivated by GR, we also tested an adaptive completion-oriented score ($\sum_d 1/r_d$, where r_d is the number of uncertain items left in d) and an equal-weight hybrid with SEAQ. These were added after the v1 freeze and evaluated on the same real episodes, so the test is exploratory. On real data the completion score loses to SEAQ by 0.156–0.223 AUC, and the hybrid by 0.002–0.005 (all CIs exclude zero). On the fan-out benchmark both tie with SEAQ, and neither repairs the high-fan-out failure. SEAQ stays the expected-resolution policy.

H. System prototype: CQP on DuckDB

To test the operator as database functionality and not only as Python arrays, we built CQP, a certified decision-query processor whose entire state lives in SQL tables in DuckDB 1.1.3. These tables hold evidence intervals, registered queries and per-entity thresholds, materialised decisions $(L, U, \text{state}, \text{emitted})$, and decision, retraction, contradiction and acquisition logs. Ingestion is one SQL pipeline per batch. It first detects contradictions by joining the batch against stored evidence and testing interval disjointness. It then builds a per-slot delta $(\Delta\ell, \Delta h)$ and applies it with one `UPDATE ...FROM` to every stock and roll-up window containing

the slot. Finally it computes state transitions, emissions and retractions; each retraction records whether a contradiction in the same batch touched the window. SEAQ is a single SQL aggregation over (uncertain item $\times$ undetermined window), and the exact planner of Theorem 2 reads its slots and ranges from SQL. `explain()` reports, for an insufficient decision, its gaps to both outcomes and its widest contributing items.

Tests (`cqp/test_cqp.py`, 9/9 pass in a fresh Python 3.12 virtual environment with pinned dependencies). The tests check that (1) incremental state equals full SQL window-function recomputation after random batches; (2) states match the reference semantics; (3) no certified decision is wrong under sound bounds; (4) a disjoint restatement is detected and every resulting retraction is flagged as contradiction-caused; (5) sound narrowing never causes a retraction (Prop. 1(b)); (6) the SQL SEAQ scores equal the NumPy reference on every item; (7) every decision counted by the exact plan is resolved under 50 random outcomes of the planned probes; (8) the plan is optimal against brute force on a small instance; and (9) the three engine modes below produce identical states and logs. **Demo** (`cqp/demo.py`) is deterministic and needs no external data. It covers two sources and three stocks in one industry, with missing and approximate evidence, insufficient alerts, SEAQ acquisition, a guaranteed-resolution plan, and a late restatement (a trade bust). The restatement produced 5 retractions, all flagged; the demo then checks the results: no certified decision is wrong against the final truth, and the incremental and full states agree to 2×10^{-13}.

Real-panel benchmark with equivalent functionality (`cqp/bench.py`, `cqp/bench_repeat.sh`, Table VIII). CQP has three engine modes with *identical observable semantics*: the same contradiction check, states, emissions, retractions and logs. In *full* mode the bounds are recomputed from scratch by SQL window functions inside the same transaction. In *inc_scan* mode the bounds are maintained by deltas and the transition step scans all decisions; in *inc* mode the transition step is restricted to the decisions touched by the delta. Test (9) checks the equivalence on randomized batches with contradictions. Each mode ran 5 times (Fig. 3), every run in a fresh process with a fresh database, with the mode order rotated across runs. Batches, engine, data layout, query set and machine were identical. We treat the run, not the batch, as the unit of replication. All 15 runs produced identical state digests and log counts. Delta maintenance is 2.30–2.37$\times$ faster than equivalent full recomputation in total time (2.30–2.44$\times$ at the median batch), and the range across runs is narrow. The affected-only and scan transition steps are indistinguishable (p50 398 vs. 404 ms). Restricting transitions saves nothing here because each changed slot feeds up to 20 windows per query, so a large share of decisions is touched per batch. Hence the $O(\deg)$ update bound does not by itself yield an end-to-end gain beyond the delta-versus-recompute difference. A first, single, 120-day run under `DOUBLE` arithmetic gave p50 540/620/1046 ms (scan/affected/full) and a p95 for affected-only (1186 ms) above full recomputation

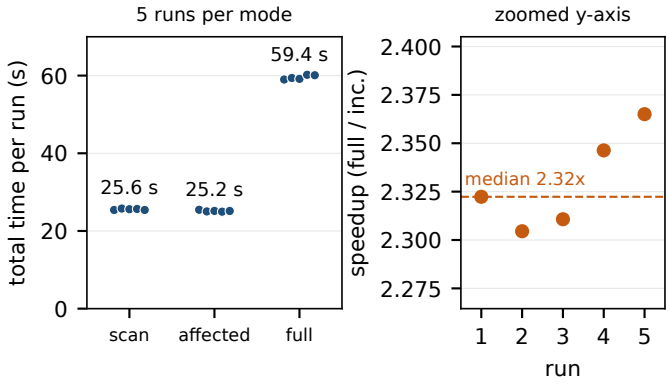


Fig. 3. Repeated DuckDB benchmark (60 days, 8 queries, 5 runs per mode, each a fresh process and database). Left (modes: incremental scan, incremental affected-only, full recomputation): total time of every run (dots) and observed min–max range (bar), axis from 0, label = median. Right: paired speedup of full recomputation over incremental (scan) per run; the y-axis is zoomed and does not start at 0. Data: `results/cqp_bench_60d_*_rep*.json`.

(1119 ms). We report it as superseded, single-run evidence (`results/cqp_bench_120d_*.json`). The gains over plain SQL are therefore the semantics (emission, retraction attribution, abstention reasons) and a consistent $\sim$2.3$\times$ latency advantage at this scale; we did not test other scales or engines.

Numerical soundness. The first version stored bounds as `DOUBLE`. Incremental and full bounds then differed by up to 10^{-4}, because approximate readings produce non-integer bounds, and floating-point summation is order-dependent. Near a threshold such a drift could flip a certified state, so CQP now certifies in *exact integer arithmetic*. Traded volumes are integers. An evidence interval $[\ell, h]$ is therefore replaced by its integer hull $[\lceil \ell \rceil, \lfloor h \rfloor]$, which is sound because the true value is an integer in $[\ell, h]$ and is never looser. A threshold θ is replaced by $\lfloor \theta \rfloor$, which is exact for integer sums: $L > \theta \iff L > \lfloor \theta \rfloor$ and $U \le \theta \iff U \le \lfloor \theta \rfloor$. All bounds are `BIGINT`, so incremental and full recomputation now agree *exactly*, and certification involves no rounding. Evidence whose integer hull is empty (for example an "exact" non-integer volume) is rejected as a violation of the domain assumption. A threshold is taken at its exact binary value: $2 \cdot 10^{12} + 0.999999$ is the double $2 \cdot 10^{12} + 1$, and a test documents this. A separate suite (`cqp/test_numeric.py`, 7 tests) checks equality at the lower and upper bounds, one integer step on either side, fractional thresholds with outward rounding, values near zero, magnitudes of 10^{12}, and 30 batches of updates with contradictions in all three engine modes. It checks every state against an independent Python-integer reference and requires exact agreement. Together with the 9 engine tests, 16/16 pass in a fresh environment. Non-integer domains would need outward-rounded interval arithmetic, which we do not implement.

Theory-to-system contract. Prop. 1 corresponds to `engine.ingest` and tests (3)–(5). Lemma 1 and Thm. 2 correspond to `acquire_sql.gr_plan` and

TABLE VIII
REPEATED EQUIVALENT-FUNCTIONALITY BENCHMARK ON DUCKDB 1.1.3, EXACT INTEGER ARITHMETIC (398 STOCKS, 60 DAYS, 8 QUERIES, 81,092 DECISIONS, 63 DAILY BATCHES). 5 RUNS PER MODE; EACH RUN IS A FRESH PROCESS AND A FRESH DATABASE, AND THE MODE ORDER IS ROTATED ACROSS RUNS. ENTRIES: MEDIAN OVER RUNS [MIN–MAX OVER RUNS] OF THE PER-RUN STATISTIC. ALL 15 RUNS PRODUCED IDENTICAL STATE DIGESTS AND LOG COUNTS. SPEEDUP OF TOTAL TIME, FULL OVER INCREMENTAL-SCAN: 2.32 [2.30–2.37].

Mode	p50 (ms)	p95 (ms)	p99 (ms)	total (s)
Incremental, transition scan	404 [396–406]	675 [656–684]	706 [688–707]	25.6 [25.4–25.8]
Incremental, affected-only	398 [391–409]	659 [646–683]	701 [655–703]	25.2 [25.0–25.5]
Full SQL recompute, same semantics	950 [931–968]	1051 [1040–1059]	1079 [1069–1093]	59.4 [59.0–60.2]

tests (7)–(8), and Thm. 1 explains why roll-ups fall back to the ILP (`run_rollup.py`). The SEAQ score is `acquire_sql.SEAQ_SQL`, checked by test (6).

I. Limits and threats to validity

Soundness is conditional: certification is only as good as the declared bounds. Domain bounds at $3\times$ the calibration maximum were violated for some items, and then decisions can be certified wrongly (Table I, Section V-D). Contradiction detection catches a violation only once disjoint evidence arrives. *Wide bounds are expensive:* with uninformative domain bounds only about 20% of decisions certify without probing. Tighter, validated bounds (e.g. from circuit limits or historical quantiles) would help but would weaken the guarantee. *Prototype scope:* CQP is an external SQL-backed operator on DuckDB, not an engine-internal operator. Its incremental path is about $2.3\times$ faster than equivalent full recomputation on one panel size and one engine; restricting transitions to affected decisions gives no further gain (Section V-H). *Scope of operators:* we evaluate SUM thresholds only. Joins with uncertain keys and non-monotone aggregates (AVG with uncertain counts) are not covered. *Sharing:* the measured benefit of sharing is small on real data and does not grow with fan-out in the controlled benchmark (Table V). *Costs:* unit costs in the main table; probe costs in practice depend on the source. *Statistics:* 8–12 episodes per cell. The CIs are over episodes for a single panel and seed family, and the stream replay is a single seed. *No worst-case guarantee* for SEAQ (Theorem 1 suggests none is cheap). *Generality:* the formulation is domain-agnostic (sensor alerts, SLA checks, fraud rules), but we evaluated only volume alerts on Indian equities plus synthetic data.

VI. WHAT IS AND IS NOT NEW

Known, reused: the certified semantics (certain answers [1], [2]; attribute bounds as in AU-DBs [4]); delta-based maintenance [12]; the gap-decomposition and min-plus convolution techniques inside the planner; the 0-1 ILP formulation; and Densest-k-Subgraph hardness [10]. *New formulation: guaranteed-resolution* (GR) acquisition for many threshold decisions sharing interval evidence. It differs from TRAPP [7] (one aggregate, a width target), from probing [8] and QOCO [9] (single queries or answer sets), and from stochastic Boolean function evaluation [11] (one function, expected cost). Budgeted maximum coverage does not apply directly, because a decision is "covered" only by *all* of its uncertain inputs (Lemma 1), which makes the objective a conjunctive, DkS-like one. *New results:* Lemma 1 and its consequences; NP-hardness of adaptive and non-adaptive GR (Thm. 1); strong polynomiality of GR for window workloads (Thm. 2), together with the line drawn by roll-ups; and the empirical finding that heuristics, including our own SEAQ, reach only 52–54% (best case) of the GR optimum at low budgets. *New system:* CQP, with contradiction-attributed retractions, SQL-native SEAQ and the planner, and three engine modes proven observably equivalent by test. *Not new:* SEAQ's score is a normalised-width heuristic. Its value is empirical (Table II), and the fan-out benchmark shows where that value comes from.

VII. RELATED WORK

Incomplete and uncertain data. Certain answers and Codd-table semantics [1], SQL's three-valued logic versus certainty [2], consistent query answering [13], probabilistic databases [5], compact world-set representations [6], and UA-/AU-DBs with attribute-level bounds [3], [4] define *what* is certain. Our semantics is a special case of these. Our contribution concerns maintaining it for windowed threshold decisions and *acquiring* evidence to resolve them. *Precision-aware refresh and probing.* TRAPP [7] and adaptive filters [14] choose which cached values to refresh so that a single (continuous) aggregate meets a precision bound. Cheng et al. [8] evaluate and probe imprecise values, and QOCO [9] asks oracles to clean answers of a given query. We differ in the objective (number of resolved *threshold decisions* rather than the width of one answer) and in sharing an input across many decisions. Widest and Local are our implementations of their single-query principle, and we show they are dominated in this setting. *Stream processing.* CQL [15] defines windowed continuous queries. The Dataflow model [16] handles lateness with watermarks and retractions, which corresponds to our WAIT and POINT engines. Distributed threshold monitoring with safe zones [17] minimises communication for one global function under exact local values. Incremental view maintenance [12] underlies our delta updates. *Provenance and cleaning.* Why-provenance [18] and semiring provenance [19] explain answers. ActiveClean [20] and stale-view cleaning [21] prioritise cleaning for model or view accuracy, and online aggregation [22] trades accuracy for time. *Theory.* Stochastic Boolean function evaluation [11] studies adaptive evaluation of one threshold function, and Densest-k-Subgraph [10] gives our hardness result.

VIII. Reproducibility

Prototype: `pip install -r requirements.txt` (pinned versions: Python 3.12, DuckDB 1.1.3, NumPy 2.2.6, SciPy 1.15.3), then `python -m pytest cqp/test_cqp.py`, `python cqp/demo.py` and `bash cqp/bench_repeat.sh` (5 runs × 3 modes, 60 days; produces Table VIII and Fig. 3 via `cqp/make_repeat_table.py` and `src/make_graphs.py`; `python cqp/bench.py 120 inc` reproduces the superseded single run), `python -m pytest cqp/test_numeric.py`. We ran these in a freshly created virtual environment and they completed as reported. A Dockerfile is included but was *not* built or tested: the Docker CLI is installed but the daemon was not running (`docker info`: cannot connect), so we claim no container reproducibility. DuckDB 2.0.0 under Python 3.14 raised an internal assertion ("dict-surviving cache") in the delta-update query on the full panel but not on small inputs. We did not isolate a minimal reproduction, and we pin DuckDB 1.1.3, the only version tested end to end. *Algorithms and tables:* The artifact contains `src/` (operator `cert.py`, policies `acquire.py`, workloads `workload.py`, experiments `run_acq.py`, `run_stream.py`, number generation `make_numbers.py`) and `DEV_LOG.md` with the freeze and SHA-256 hashes. All numbers in this paper are macros generated from `results/*.json`. Commands: `python run_acq.py real 8 --tag main`; `python run_acq.py synth 12 --seed 3 --tag main`; `python run_stream.py 7`; `python run_fanout.py`; `python wcdp.py` (DP vs. brute force); `python run_wc.py`; `python make_wc.py`; `python wcdp_sp.py`; `python run_rollup.py`; `python make_rollup.py`; `python verify_theory.py`; `CAP_MULT=10 python run_stream.py 7 0.3 0.3 0.0` (and the other three soundness settings); `python make_numbers.py`; `python make_table.py`; `python make_extra.py`. The exchange files are public, but we do not redistribute them; the original download scripts and checksums are included. Hardware: a 12-thread Windows 11 laptop with 7.7 GB RAM, Python 3.14, NumPy 2.5, SciPy 1.18. Every run in the paper completes in minutes.

IX. Conclusion

Treating threshold queries as *decisions* over bounded evidence separates what a monitoring engine can certify from what it is guessing. The certified state is cheap to maintain incrementally. Resolving the rest is combinatorially hard in the worst case, but a shared, normalised progress score resolves decisions with substantially less acquisition than single-query refresh rules or myopic VOI. For window workloads, guaranteed-resolution plans can be optimised exactly and fast, and heuristics fall far short of the optimum. Two further findings matter for practice. Certified errors arise only from unsound evidence (violated bounds and restatements), and bound width trades coverage against safety. Shared acquisition helps at low-to-moderate fan-out but not at high fan-out, where fetching everything is as good. The open problems are validated tight bounds, operators beyond SUM, and an acquisition objective that actually benefits from sharing.

Acknowledgment and AI-Generated Content Disclosure

An AI assistant (Anthropic Claude) assisted with code, experiments and drafting under the authors' direction; the authors take full responsibility for the content.

References

[1] T. Imieliński and W. Lipski, Jr., "Incomplete information in relational databases," *J. ACM*, vol. 31, no. 4, pp. 761–791, 1984.

[2] L. Libkin, "SQL's three-valued logic and certain answers," *ACM Trans. Database Syst.*, vol. 41, no. 1, pp. 1:1–1:28, 2016.

[3] S. Feng, A. Huber, B. Glavic, and O. Kennedy, "Uncertainty annotated databases – a lightweight approach for approximating certain answers," in *SIGMOD*, 2019, pp. 1313–1330.

[4] S. Feng, B. Glavic, A. Huber, and O. Kennedy, "Efficient uncertainty tracking for complex queries with attribute-level bounds," in *Proc. ACM SIGMOD*, 2021.

[5] N. N. Dalvi and D. Suciu, "Efficient query evaluation on probabilistic databases," in *VLDB*, 2004, pp. 864–875.

[6] L. Antova, C. Koch, and D. Olteanu, "10^{10^6} worlds and beyond: Efficient representation and processing of incomplete information," in *ICDE*, 2007, pp. 606–615.

[7] C. Olston and J. Widom, "Offering a precision-performance tradeoff for aggregation queries over replicated data," in *VLDB*, 2000, pp. 144–155.

[8] R. Cheng, D. V. Kalashnikov, and S. Prabhakar, "Evaluating probabilistic queries over imprecise data," in *SIGMOD*, 2003, pp. 551–562.

[9] M. Bergman, T. Milo, S. Novgorodov, and W.-C. Tan, "QOCO: A query oriented data cleaning system with oracles," *PVLDB*, vol. 8, no. 12, pp. 1900–1903, 2015.

[10] U. Feige, D. Peleg, and G. Kortsarz, "The dense k-subgraph problem," *Algorithmica*, vol. 29, no. 3, pp. 410–421, 2001.

[11] A. Deshpande, L. Hellerstein, and D. Kletenik, "Approximation algorithms for stochastic boolean function evaluation and stochastic submodular set cover," in *Proc. ACM-SIAM SODA*, 2014.

[12] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, "Maintaining views incrementally," in *SIGMOD*, 1993, pp. 157–166.

[13] M. Arenas, L. E. Bertossi, and J. Chomicki, "Consistent query answers in inconsistent databases," in *PODS*, 1999, pp. 68–79.

[14] C. Olston, J. Jiang, and J. Widom, "Adaptive filters for continuous queries over distributed data streams," in *SIGMOD*, 2003, pp. 563–574.

[15] A. Arasu, S. Babu, and J. Widom, "The CQL continuous query language: Semantic foundations and query execution," *VLDB J.*, vol. 15, no. 2, pp. 121–142, 2006.

[16] T. Akidau *et al.*, "The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing," *PVLDB*, vol. 8, no. 12, pp. 1792–1803, 2015.

[17] I. Sharfman, A. Schuster, and D. Keren, "A geometric approach to monitoring threshold functions over distributed data streams," in *SIGMOD*, 2006, pp. 301–312.

[18] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *ICDT*, 2001, pp. 316–330.

[19] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *PODS*, 2007, pp. 31–40.

[20] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "ActiveClean: Interactive data cleaning for statistical modeling," *PVLDB*, vol. 9, no. 12, pp. 948–959, 2016.

[21] S. Krishnan, J. Wang, M. J. Franklin, K. Goldberg, and T. Kraska, "Stale view cleaning: Getting fresh answers from stale materialized views," *PVLDB*, vol. 8, no. 12, pp. 1370–1381, 2015.

[22] J. M. Hellerstein, P. J. Haas, and H. J. Wang, "Online aggregation," in *SIGMOD*, 1997, pp. 171–182.